Comparative Study of Parallelization Techniques for Word Counting Map Reduce Algorithm

Kuan Hsi Chu
National Yang Ming Chiao Tung University
Taiwan

Tzu Jung Chen
National Yang Ming Chiao Tung University
Taiwan

Hsiao Hsin Lin
National Yang Ming Chiao Tung University
Taiwan

ABSTRACT

This project seeks to delve into the diverse implementation strategies, performance dynamics, and development hurdles associated with various parallelization techniques applied to a common computational task: Word Counting. Our approach involves the creation of four distinct custom frameworks that emulate the MapReduce[1] paradigm, utilizing Pthreads, OpenMP, MPI, and CUDA. Each methodology embodies a unique model of parallelization: Pthreads and OpenMP facilitate multi-threaded parallelism on CPUs, MPI enables distributed computation across multiple nodes, and CUDA leverages the capabilities of GPUs for data-parallel operations. Through this comparative study, we will thoroughly examine how each framework approaches the essential Map, Shuffle, and Reduce phases of the computation process. Our analysis aims to illuminate the inherent trade-offs related to communication overhead, memory management, and scalability, providing valuable insights into the effectiveness and efficiency of each parallelization technique in the context of our core application.

1 INTRODUCTION

High-performance computing (HPC) and big data processing frequently rely on the MapReduce paradigm for its robust and scalable nature. While industrial frameworks like Hadoop and Spark abstract the parallelization details, a deep understanding of the underlying hardware and programming models is crucial for performance tuning. This project aims to strip away that abstraction by implementing the core MapReduce workflow (Map, Shuffle, Reduce) using four fundamental parallel programming models: MPI (Distributed Memory), OpenMP (Shared Memory CPU), Pthreads (Shared Memory CPU), and CUDA (Data-Parallel GPU)[2]. By comparing these four distinct implementations on the identical computational problem, we can quantitatively assess the intrinsic performance trade-offs, scalability limits, and architectural bottlenecks of each paradigm when simulating a real-world big data workload.

2 STATEMENT OF THE PROBLEM

The goal of this project is to implement the Word Count problem using multiple parallelization techniques based on MapReduce principles (Pthreads, OpenMP, MPI, and CUDA). In MapReduce, the "map" stage processes data in parallel to produce intermediate key-value pairs, while the "reduce" stage aggregates results into a final output. Implementing Word Count across these diverse architectures allows us to conduct a direct, quantitatie comparison of their performance in terms of execution time, scalability, and resource utilization. The central challenges we investigate include: evenly distributing workloads, handling word boundaries, efficiently merging local results, and minimizing communication/synchronization overhead in shared-memory, distributed, and GPU environments. This comparison will define the optimal parallel model for each phase of the MapReduce workflow.

3 PROPOSED APPROACHES

The project will implement four different parallelized MapReduce-style word count solutions. Pthreads and OpenMP: The text dataset will be divided into multiple segments. Each thread will process one segment, building a local histogram of word counts. Local histograms will then be merged into a global histogram. OpenMP provides loop-level parallelization, simplifying the implementation, while Pthreads allows finer control over thread creation and synchronization. MPI: The dataset will be distributed across multiple processes in a cluster environment. Each process counts words in its assigned portion of the dataset. MPI collective communication routines, such as MPI_Reduce or MPI_Gather, will aggregate results into a global histogram. This approach highlights distributed computation challenges and communication overhead. CUDA: The dataset will be divided into blocks assigned to GPU threads. Each thread will perform word counting on its block, storing intermediate results in shared memory. A parallel reduction algorithm will merge local histograms into a global result. Special attention will be given to handle word boundaries that span multiple blocks. Integration and Interaction: Across all approaches, a master component will coordinate data segmentation, assignment, and result aggregation, while worker components handle computation. The interactions among components involve distributing data, computing local results, and merging outputs into a global histogram. This architecture ensures consistency in results across all implementations and facilitates performance comparison.

4 LANGUAGE SELECTION

The implementations will use C/C++ as the base programming language. Pthreads and OpenMP will leverage C/C++ multi-threading capabilities for CPU parallelism. MPI will be implemented in C/C++ for distributed MapReduce coordination using message passing. CUDA will use the CUDA C API to exploit GPU parallelism. These choices provide full control over parallel computation and satisfy the requirement of not using pre-parallelized libraries, allowing us to explore implementation challenges and optimization strategies directly.

5 RELATED WORK

Word Count is commonly used as a benchmark in parallel computing research. MPI-based approaches demonstrate distributed computation efficiency, while Pthreads and OpenMP implementations provide insight into multi-threaded CPU performance. CUDA

implementations illustrate massive parallelism on GPUs. High-level frameworks like Hadoop or Spark offer MapReduce [1] abstractions. Yadav el. [3] implement Word-Count with Map Reduce Programming Using Hadoop Framework. This project builds upon these studies by creating all four MapReduce implementations from scratch, also we do the performance comparisons like [4] [5] and revealing the underlying design and parallelization challenges.

6 STATEMENT OF EXPECTED RESULTS

We expect to produce four functioning MapReduce-style Word Count implementations. Performance will be evaluated in terms of execution time, speedup, efficiency, and scalability for varying dataset sizes. Implementation challenges, including workload distribution, synchronization, communication overhead, and memory management, will be documented. The study will provide a clear comparative analysis of CPU multi-threading, distributed computing, and GPU parallelization strategies for MapReduce-style tasks.

7 TIMETABLE

The project will follow an eight-week schedule. Weeks 1–2 will focus on literature review and understanding of parallelization techniques. Weeks 3–4 will cover development and testing of Pthreads, OpenMP, and MapReduce implementations. Weeks 5–6 will be dedicated to MPI, CUDA, and OpenCL development and debugging. Week 7 will involve performance testing, data collection, and analysis. Finally, week 8 will focus on compiling results, writing the final report, and preparing the presentation.

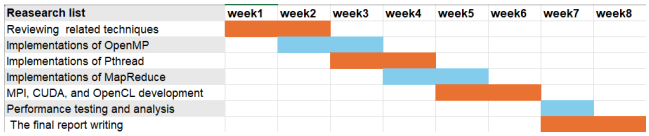


Figure 1: Gantt chart of our final project

REFERENCES

[1] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: simplified data processing on large clusters. *Commun. ACM*, 51(1):107–113, January 2008.

[2] Peter Pacheco and Matthew Malensek. *An introduction to parallel programming*. Morgan Kaufmann, 2021.

[3] Santosh Yadav and Jay Prakash. A survey on implementation of word-count with map reduce programming oriented model using hadoop framework. *SSRN Electronic Journal*, 01 2019.

[4] Zahra Mohamed and Abdulla AlQaddoumi. Comparing sequential and parallel word counting methods in finding commonly used words by different authors. pages 1–4, 2020.

[5] Sol Ji Kang, Sang Yeon Lee, and Keon Myung Lee. Performance comparison of openmp, mpi, and mapreduce in practical problems. *Advances in Multimedia*, 2015(1):575687, 2015.